

Week 04: Trees, evaluation and termination

Functional Programming in Lean

Learning goals

- ▶ Define operations on a persistent tree
- ▶ Compare ordinary and accumulator recursion
- ▶ Explain strict evaluation in Lean

Central idea

A tree operation follows the shape of the tree.

Accumulators can change the cost of a traversal.

Lean example

```
def Tree.size : Tree  $\alpha$   $\rightarrow$  Nat
  | .leaf => 0
  | .node left _ right =>
    left.size + 1 + right.size
```

What to notice

- ▶ Lean evaluates function arguments before a call
- ▶ An explicit `Unit →  $\alpha$` function can delay a computation
- ▶ A terminating recursive definition can participate in proofs

Live coding

1. Complete `Tree.size`
2. Write `reverseAcc` with a local helper
3. Compare the order of results with `List.reverse`

Pause and predict

Why is the recursive call on a subtree accepted?

Exercise connection

Open `exercises/Week04.md` after the lecture.

The complete example is `Course/Week04.lean`; the fill-in version is `Templates/Week04.lean`.